

Some useful sums:

- $\sum_{i=1}^n 2^i = 2(2^n - 1)$
- $\sum_{i=1}^n 2^i i = 2(2^n n - 2^n + 1)$
- $\sum_{i=0}^{t-1} i a^i = \frac{(t-1)a^{t+1} - t a^t + a}{(a-1)^2}$
- $\sum_{i=0}^{n-2} (n-i) = \frac{1}{2} (n-1)(n+2)$
- $\sum_{i=0}^{n-2} (n-i)^2 = \frac{1}{6} (n-1)(2n^2 + 5n + 6)$
- $\sum_{i=0}^{n-2} (n-i)^3 = \frac{1}{4} (n-1)(n+2)(n^2 + n + 2)$

Chuỗi thông dụng

16/06/2014

$$S = 1 + 2 + 3 + \dots + n = n(n+1)/2 \approx n^2/2$$

$$S = 1 + 2^2 + 3^2 + \dots + n^2 = n(n+1)(2n+1)/6 \approx n^3/3$$

$$S = 1 + a + a^2 + a^3 + \dots + a^n = (a^{n+1} - 1)/(a-1)$$

Nếu $0 < a < 1$ thì

$$S \leq 1/(1-a)$$

và khi $n \rightarrow \infty$ thì

$$S \text{ tiến về } 1/(1-a)$$

$$S = 1 + 1/2 + 1/3 + \dots + 1/n = \ln(n) + \gamma$$

Hằng số Euler $\gamma \approx 0.577215665$

$$S = 1 + 1/2 + 1/4 + 1/8 + \dots + 1/2^n + \dots \approx 2$$

Ví dụ (tt)

$$NR = \sum_{j=0}^{k-1} a^j d(b^{k-j}) = \sum_{j=0}^{k-1} 2^j \log 2^{k-j}$$

$$NR = \sum_{j=0}^{k-1} 2^j (k-j) = \sum_{j=0}^{k-1} k 2^j - \sum_{j=0}^{k-1} j 2^j$$

$$NR = O(k \sum_{j=0}^{k-1} 2^j) = O(k \frac{2^k - 1}{2 - 1})$$

$$NR = O(k 2^k) = O(n \log n) > n = NTN$$

$$T(n) = O(n \log n)$$

exponentiation.

Answer: (penalty regime: 33.33, 66.67, ... %)

$$\begin{aligned}
 T(n) &= 5T(n/4) + n \\
 &= 5 T(n/4^1) + n \\
 &= 25 T(n/4^2) + n + n \cdot 5/4 \\
 &= 125 T(n/4^3) + n + n \cdot 5/4 + n \cdot (5/4)^2 \\
 &= 625 T(n/4^4) + n + n \cdot 5/4 + n \cdot (5/4)^2 + n \cdot (5/4)^3 \\
 &= \dots \\
 &= 5^t T(n/4^t) + \sum_{i=0}^{t-1} n \cdot (5/4)^i \\
 &= \dots \\
 &= n^{\log(4,5)} T(1) + n \cdot ((5/4)^{\log(4,n)} - 1) / (5/4 - 1) \\
 &= O(n^{\log(4,5)}), \text{ e.g., } O(n \log(n))
 \end{aligned}$$

Using substitution method, find the pattern of the below recurrence relation:

$$T(n) = 22T(n/3) + n^3$$

$$T(1) = 1$$

Convention

- Use $\log(2, n)$ for $\log_2(n)$
- +, -, *, /, and ^ for addition, subtraction, multiplication, division and exponentiation.

Answer: (penalty regime: 33.33, 66.67, ... %)

$$\begin{aligned}
 T(n) &= 22T(n/3) + n^3 \\
 &= 22 T(n/3^1) + n^3 \\
 &= 22^2 T(n/3^2) + n^3 + n^3(22/3^3)^1 \\
 &= 22^3 T(n/3^3) + n^3 + n^3(22/3^3)^1 + n^3(22/3^3)^1 \\
 &= 22^4 T(n/3^4) + n^3 + n^3(22/3^3)^1 + n^3(22/3^3)^1 \\
 &= \dots \\
 &= 22^t T(n/3^t) + \sum_{i=0}^{t-1} n^3(22/3^3)^i \\
 &= \dots \\
 &= n^{\log(3,22)} T(1) + n^3 * ((22/3^3)^{\log(3, n)} - 1) / (22/3) \\
 &= O(n^3), \text{ e.g., } O(n^3 \log(n))
 \end{aligned}$$

Using substitution method, find the pattern of the below recurrence relation:

$$T(n) = 2T(n - 1) + 1$$

$$T(1) = 1$$

Convention

- Use $\log(2, n)$ for $\log_2(n)$
- +, -, *, /, and ^ for addition, subtraction, multiplication, division and exponentiation.

Answer: (penalty regime: 33.33, 66.67, ... %)

$$\begin{aligned}
 T(n) &= 2T(n - 1) + 1 \\
 &= 2 T(n - 1) + 1 \\
 &= 4 T(n - 2) + 3 \\
 &= 8 T(n - 3) + 7 \\
 &= 16 T(n - 4) + 15 \\
 &= \dots \\
 &= 2^t T(n - t) + \sum_{i=0}^{t-1} 2^i \\
 &= \dots \\
 &= 2^{(n-1)} T(1) + 2^{(n-1)} - 1
 \end{aligned}$$

Using substitution method, find the pattern of the below recurrence relation:

$$T(n) = 5T(n - 1) + 1$$

$$T(1) = 1$$

Convention

- Use $\log(2, n)$ for $\log_2(n)$
- +, -, *, /, and ^ for addition, subtraction, multiplication, division and exponentiation.

Answer: (penalty regime: 33.33, 66.67, ... %)

$$\begin{aligned}
 T(n) &= 5T(n - 1) + 1 \\
 &= 5 T(n - 1) + 1 \\
 &= 25 T(n - 2) + 6 \\
 &= 125 T(n - 3) + 31 \\
 &= 625 T(n - 4) + 156 \\
 &= \dots \\
 &= 5^t T(n - t) + \sum_{i=0}^{t-1} 5^i \\
 &= \dots \\
 &= 5^{(n-1)} T(1) + (5^{(n-1)} - 1)/4
 \end{aligned}$$

Using substitution method, find the pattern of the below recurrence relation:

$$T(n) = T(n - 1) + n$$

$$T(1) = 1$$

Convention

- Use $\log_2(n)$ for $\log_2(n)$
- +, -, *, /, and ^ for addition, subtraction, multiplication, division and exponentiation.

Answer: (penalty regime: 33.33, 66.67, ... %)

$T(n) = T(n - 1) + n$

= $T(n - 1) +$

= $T(n - 2) +$

= $T(n - 3) +$

= $T(n - 4) +$

= ...

= $T(n - t) + \sum_{i=0}^{t-1} n - i$

= ...

= $T(1) +$

Using substitution method, find the pattern of the below recurrence relation:

$$T(n) = T(n - 1) + n^2$$

$$T(1) = 1$$

Convention

- Use $\log_2(n)$ for $\log_2(n)$
- +, -, *, /, and ^ for addition, subtraction, multiplication, division and exponentiation.

Answer: (penalty regime: 33.33, 66.67, ... %)

$T(n) = T(n - 1) + n^2$

= $T(n - 1) +$

= $T(n - 2) +$

= $T(n - 3) +$

= $T(n - 4) +$

= ...

= $T(n - t) + \sum_{i=0}^{t-1} (n-i)^2$

= ...

= $T(1) +$

$$T(n) = 2T(n/2) + \log(2, n)$$

$$= 2 T(n/2^1) + \log(2, n)$$

$$= 4 T(n/2^2) + \log(n) + 2*\log(n/2)$$

$$= 8 T(n/2^3) + \log(n) + 2*\log(n/2) + 4*\log(n/4)$$

$$= 16 T(n/2^4) + \log(n) + 2*\log(n/2) + 4*\log(n/4) + 8*\log(n/8)$$

= ...

$$= 2^t T(n/2^t) + \sum_{i=0}^{t-1} 2^i \log(n/2^i)$$

= ...

$$= n T(1) + 2*n - \log(n) - 2$$

Using substitution method, find the pattern of the below recurrence relation:

$$T(n) = 3T(n/5) + \log(5, n)$$

$$T(1) = 1$$

Convention

- Use $\log(2, n)$ for $\log_2(n)$
- +, -, *, /, and ^ for addition, subtraction, multiplication, division and exponentiation.

Answer: (penalty regime: 33.33, 66.67, ... %)

$$T(n) = 3T(n/5) + \log(5, n)$$

$$= 3 T(n/5^1) + \log(5, n)$$

$$= 9 T(n/5^2) + \log(5, n) + 3^1 \log(5, n/5^1)$$

$$= 27 T(n/5^3) + \log(5, n) + 3^1 \log(5, n/5^1) + 3^2 \log(5, n/5^2)$$

$$= 81 T(n/5^4) + \log(5, n) + 3^1 \log(5, n/5^1) + 3^2 \log(5, n/5^2) + 3^3 \log(5, n/5^3)$$

= ...

$$= 3^t T(n/5^t) + \sum_{i=0}^{t-1} (3^i) \log(5, n/5^i)$$

= ...

$$= n^{\log(5,3)} T(1) + (3 * 3^{\log(5, n)} - \log(5, n) * (3 - 1) - 3) / (3 - 1)^2$$

Using substitution method, find the pattern of the below recurrence relation:

$$T(n) = 5T(n/2) + \log(2, n)$$

$$T(1) = 1$$

Convention

- Use $\log(2, n)$ for $\log_2(n)$
- +, -, *, /, and ^ for addition, subtraction, multiplication, division and exponentiation.

Answer: (penalty regime: 33.33, 66.67, ... %)

$$\begin{aligned}
 T(n) &= 5T(n/2) + \log(2, n) \\
 &= 5 T(n/2^1) + \log(2, n) \\
 &= 25 T(n/2^2) + 5*\log(2, n/2) + \log(2, n) \\
 &= 125 T(n/2^3) + 25*\log(2, n/4) + 5*\log(2, n/2) + \log(2, n) \\
 &= 625 T(n/2^4) + 125*\log(2, n/8) + 25*\log(2, n/4) + 5*\log(2, n/2) + \log(2, n) \\
 &= \dots \\
 &= 5^t T(n/2^t) + \sum_{i=0}^{t-1} (5^i)*\log(2, n/2^i) \\
 &= \dots \\
 &= n^{\log(2, 5)} T(1) + (5 * 5^{\log(2, n)} - \log(2, n)*(5 - 1) - 5) / (5 - 1)^2
 \end{aligned}$$

Check

Using substitution method, find the pattern of the below recurrence relation:

$$T(n) = 2T(n/2) + n * \log(2, n)$$

$$T(1) = 1$$

Convention

- Use $\log(2, n)$ for $\log_2(n)$
- +, -, *, /, and ^ for addition, subtraction, multiplication, division and exponentiation.

Answer: (penalty regime: 33.33, 66.67, ... %)

$$T(n) = 2T(n/2) + n * \log(2, n)$$

$$= 2 T(n/2^1) + n * \log(2, n)$$

$$= 4 T(n/2^2) + n * \log(2, n/2) + n * \log(2, n)$$

$$= 8 T(n/2^3) + n * \log(2, n/2^2) + n * \log(2, n/2^1) + n * \log(2, n)$$

$$= 16 T(n/2^4) + n * \log(2, n/2^3) + n * \log(2, n/2^2) + n * \log(2, n/2^1) + n * \log(2, n)$$

= ...

$$= 2^t T(n/2^t) + \sum_{i=0}^{t-1} n * (2/2)^i * \log(2, n/2^i)$$

= ...

$$= n T(1) + n/2 * (\log(n) * (\log(n) + 1))$$

Using substitution method, find the pattern of the below recurrence relation:

$$T(n) = 3T(n/2) + n * \log_2(n)$$

$$T(1) = 1$$

Convention

- Use $\log_2(n)$ for $\log_2(n)$
- +, -, *, /, and ^ for addition, subtraction, multiplication, division and exponentiation.

Answer: (penalty regime: 33.33, 66.67, ... %)

$$T(n) = 3T(n/2) + n * \log_2(n)$$

$$= 3 T(n/2^1) + n * \log_2(n)$$

$$= 9 T(n/2^2) + n * \log_2(n) + n * (3/2)^1 * \log_2(n/2^1)$$

$$= 27 T(n/2^3) + n * \log_2(n) + n * (3/2)^1 * \log_2(n/2^1) + n * (3/2)^2 * \log_2(n/2^2)$$

$$= 81 T(n/2^4) + n * \log_2(n) + n * (3/2)^1 * \log_2(n/2^1) + n * (3/2)^2 * \log_2(n/2^2) + n * (3/2)^3 * \log_2(n/2^3)$$

= ...

$$= 3^t T(n/2^t) + \sum_{i=0}^{t-1} n * (3/2)^i * \log_2(n/2^i)$$

= ...

$$= n^{\log_2(3)} T(1) + n * ((3/2)^{(1+\log_2(n))} - (3/2 - 1) * \log_2(n) - 3/2) / (3/2 - 1)^2$$

SELECT

```
#include <stdio.h>
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int Select(vector<int> A, int k) {
```

```
    // Nếu len(A) <= 10 thì sắp xếp A và trả về A[k-1]
```

```
    if ( A.size() <= 10 ) {
```

```
        sort(A.begin(), A.end());    //Sắp xếp vector A tăng dần
```

```
        return A[k - 1] ;
```

```
    }
```

```
    // Chọn pivot, hãy viết mã để chọn pivot là phần tử A[len(A)/2]
```

```

int p = A.size()/2          ; //p là vị trí chính giữa mảng
int pivotVal = A[p]        ;

// Phân hoạch A thành L, pivotVal và R
vector<int> L, R;
for (int i = 0; i < A.size(); i++) {
    if(A[i] < pivotVal)
        L.push_back(A[i]);
    if(A[i] > pivotVal)
        R.push_back(A[i]);

}

if (k == L.size() + 1)      // k = len(L) + 1
    return pivotVal        ;
if (k < L.size() + 1)      // k < len(L) + 1
    return Select(L,k)      ;
return Select(R,k - L.size() - 1) ;      // k > len(L) + 1
}

//Bước 4: Kiểm thử
int main() {
    int n, x;
    vector<int> A;
    cin >> n;
    A.resize(n); //Thay đổi kích thước của A về n phần tử
    for (int i = 0; i < n; i++) {
        cin >> A[i];
    }

    for (int k = 1; k <= n; k++)
        cout << "Select(A, " << k << ") = " << Select(A, k) << endl;
    return 0;
}

```

```

#include <stdio.h>
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int Select(vector<int> A, int k) {
    // Nếu len(A) <= 10 thì sắp xếp A và trả về A[k-1]
    if ( A.size() <= 10 ) {
        sort(A.begin(), A.end());    //Sắp xếp vector A tăng dần
        return A[k - 1] ;
    }
    // Chọn pivot, hãy viết mã để chọn pivot theo phương pháp "fancy pivot"
    int n = A.size();
    vector<int> B;
    for (int i = 0; i < n; i += 5) {
        int group_size = i + 5 <= n ? 5 : n - i;
        //Tìm phần tử trung vị của A[i], A[i+1], ..., A[i + group_size - 1] và đưa vào B
        //Sử dụng vector temp chứa các phần tử A[i], A[i+1], ..., A[i + group_size - 1]
        //Gọi hàm sort() để sắp xếp các phần tử của temp
        //Lấy phần tử chính giữa của temp, tức temp[len(temp)/2], đưa vào B
        vector<int> temp;

        while(group_size>0){
            temp.push_back(A[i + group_size - 1]);
            group_size--;
        }
        sort(temp.begin(), temp.end());
        B.push_back(temp[temp.size()/2]);
    }
    // Gọi Select(B, len(B)/2) để tìm pivotVal
    int pivotVal = Select(B, B.size()/2) ;

    // Kiểm tra pivotVal
    cout << "pivot value = " << pivotVal << endl;

```

```

// Phân hoạch A thành L, pivotVal và R
vector<int> L, R;
for (int i = 0; i < A.size(); i++) {
    if(A[i] < pivotVal)
        L.push_back(A[i]);
    if(A[i] > pivotVal)
        R.push_back(A[i]);
}

if (k == L.size() + 1)    // k = len(L) + 1
    return pivotVal      ;
if (k < L.size() + 1)    // k < len(L) + 1
    return Select(L,k)    ;
return Select(R,k - L.size() - 1) ;    // k > len(L) + 1
}

//Bước 4: Kiểm thử
int main() {
    int n, k;
    vector<int> A;
    cin >> n >> k;
    A.resize(n); //Thay đổi kích thước của A về n phần tử
    for (int i = 0; i < n; i++) {
        cin >> A[i];
    }

    int ret = Select(A, k);
    cout << "Select(A, " << k << ") = " << ret << endl;
    return 0;
}

```

INSERTION SORT

```

//Cài đặt thuật toán sắp xếp xen
void insertionSort(int A[], int n) {

```

```

for (int i = 1; i < n; i++) {
    //Xen phần tử A[i] vào vị trí thích hợp
    int current = A[i];
    int j = i - 1;
    while(j >= 0 && A[j] > current){
        A[j + 1] = A[j];
        j--;
    }
    A[j + 1] = current;
    //In các phần tử trong mảng A để kiểm tra việc xen
    for (int j = 0; j < n; j++)
        cout << A[j] << ' ';
    cout << endl;
}
}

```

MERGE SORT

```

vector<int> mergeSort(vector<int> A) {
    int n = A.size();
    if (n <= 1)
        return A;

    int m = n/2;
    vector<int> L = mergeSort(vector<int>(A.begin(), A.begin() + m));
    vector<int> R = mergeSort(vector<int>(A.begin() + m, A.end()));
    vector<int> K;
    //Trộn 2 danh sách L và R thành danh sách K
    auto l = L.begin();
    auto r = R.begin();
    while(l != L.end() && r != R.end()){
        int temp;

        if(*l < *r){
            temp = *l;

```

```

        K.push_back(temp);
        l++;
    } else{
        temp = *r;
        K.push_back(temp);
        r++;
    }
}
while(l!= L.end()){
    int temp = *l;
    K.push_back(temp);
    l++;
}
while(r!=R.end()){
    int temp = *r;
    K.push_back(temp);
    r++;
}

return K;
}

```

```

int find_min(int i, int j, int A[]) {
    if (i == j) //A has an only one element, don't split
        return A[i];
    //Divide: split A into two halves
    int m = (i + j)/2; //left half: A[i], A[i+1], ..., A[m]
                        //right half: A[m+1], A[m+2], ..., A[j]
    //Conquer two halves: recursively call find_min on two halves
    int min_left = find_min(i, m, A);
    int min_right = find_min(m + 1, j, A);
    //Combine: compute the minimum of min_left and min_right
    return min(min_left, min_right);
}

```